

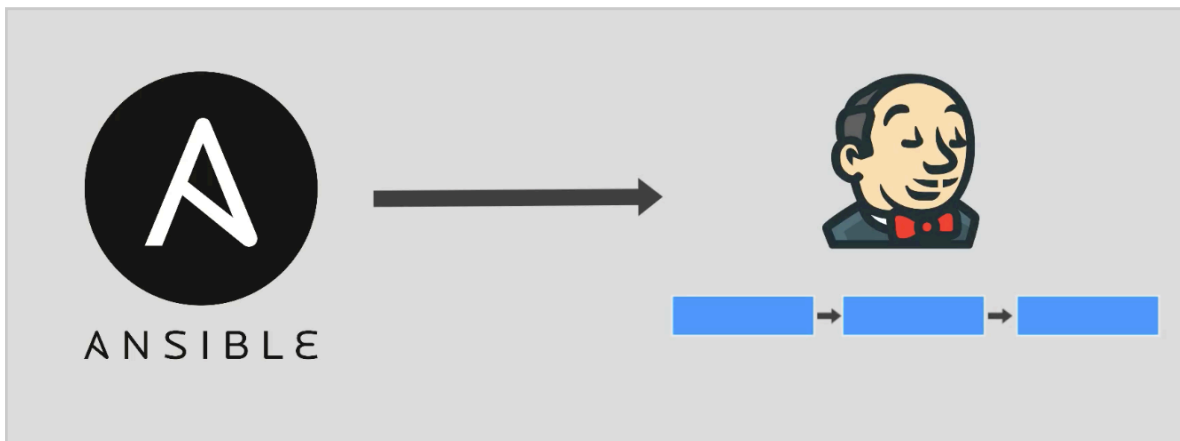
Module 15 - chapter 21

Project: Run Ansible from Jenkins Pipeline - Part 1

Ansible Integration in Jenkins

Overview

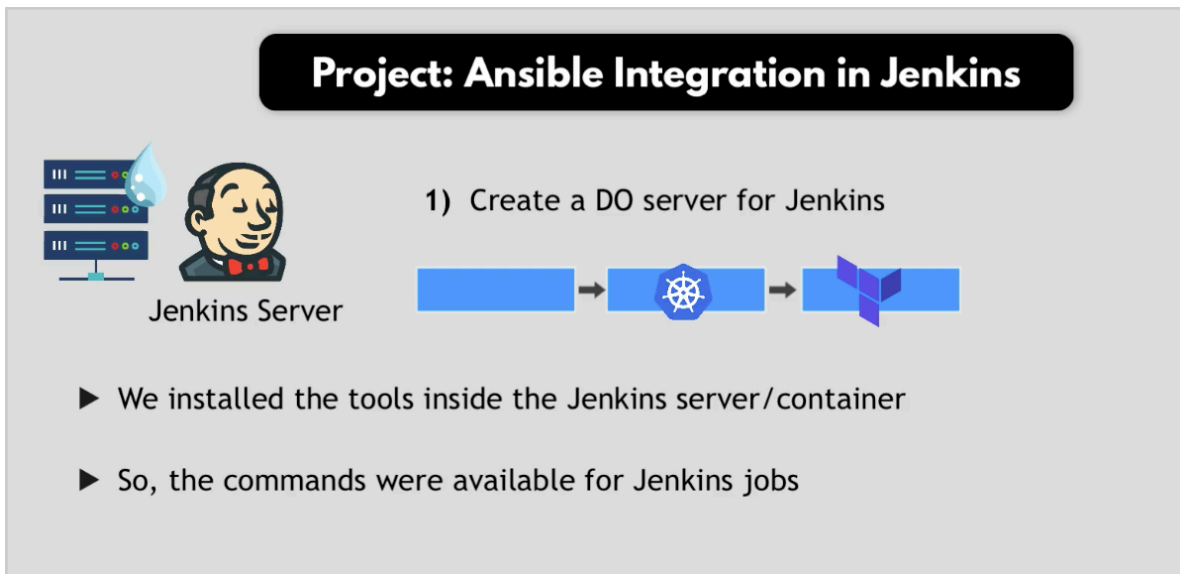
How to execute ansible playbooks from Jenkins Pipeline:



Implementation steps:

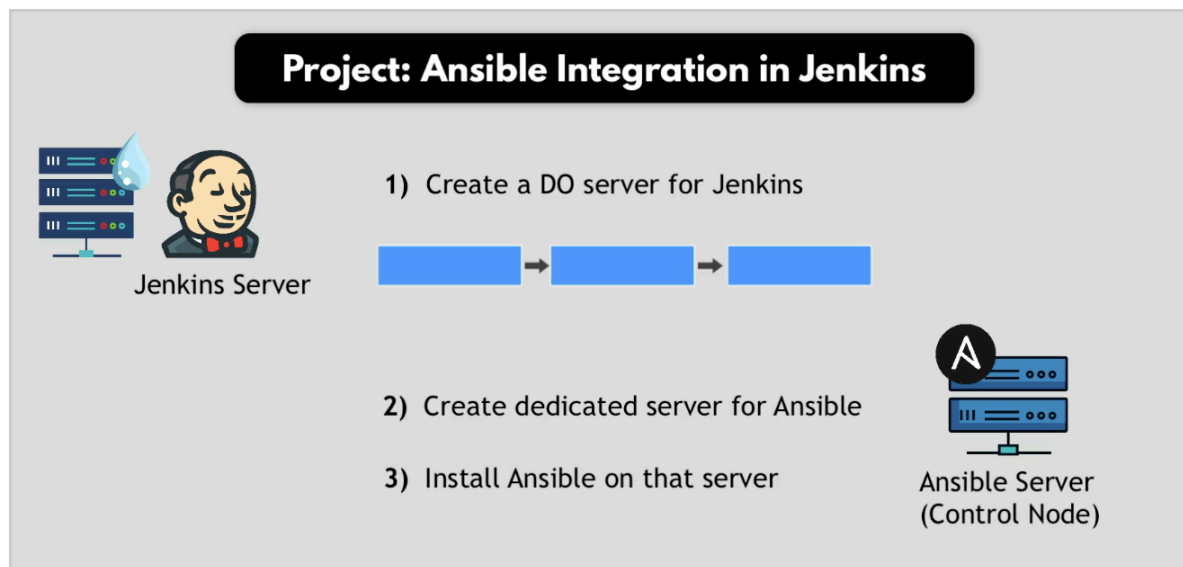
1. Create a DO (DigitalOcean) droplet

Previously we learn that if we wanted to use a tool in Jenkins we need to install it there first for example kubectl.



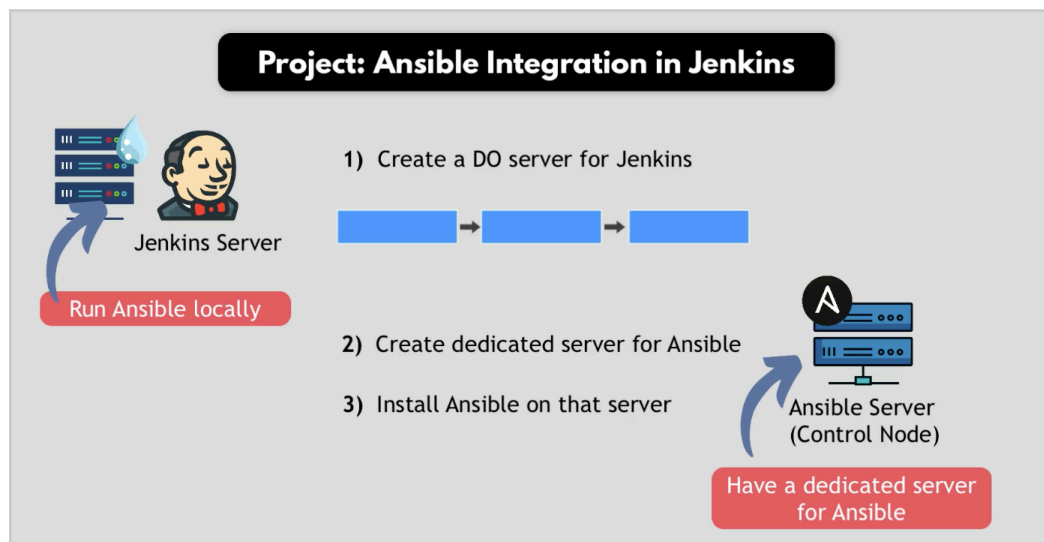
But for ansible it will be different, instead of installing ansible in the Jenkins container and making it available in the Jenkins Server we will create a dedicated server for ansible and install ansible there.

2. Create dedicated server for ansible
3. Install ansible on that server

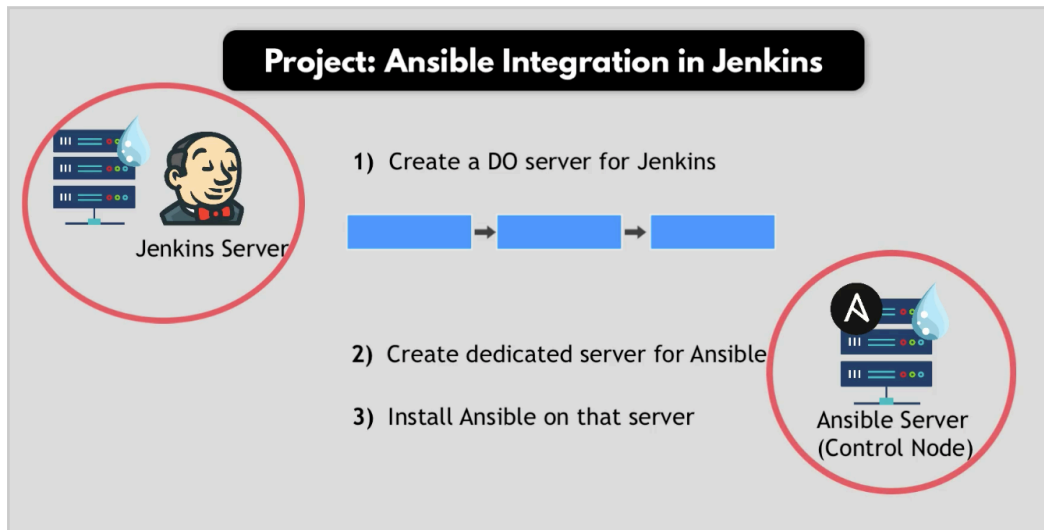


We can run ansible locally for example in our laptop or in Jenkins directly.

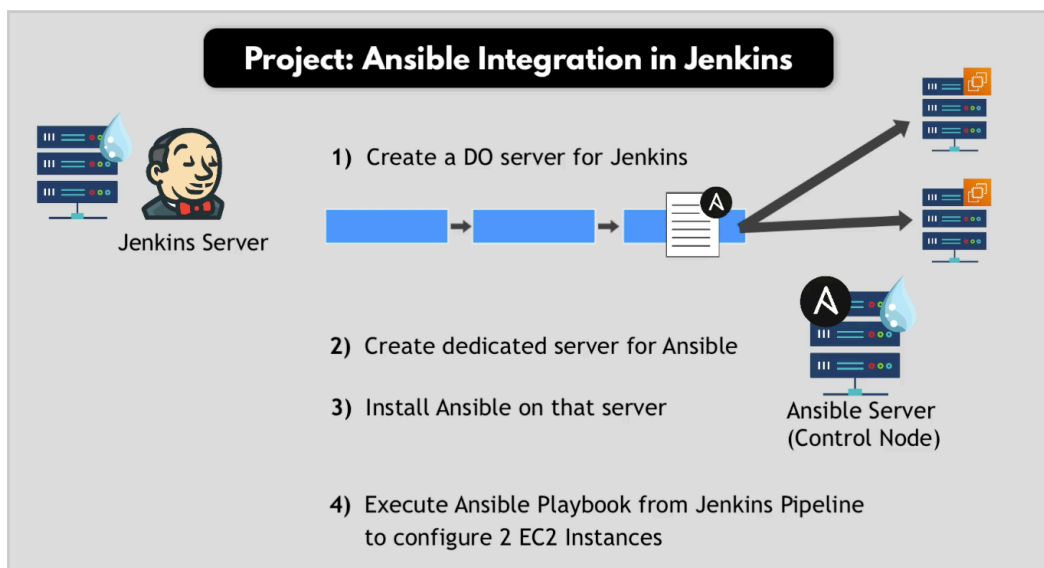
Or as a common practice we will have a dedicated server for ansible and install ansible there and execute ansible commands from that server.



So we will have separate droplets



4. Then we want to execute an Ansible playbook from Jenkins Pipeline to configure two EC2 instances, it will install docker and docker-compose on the EC2 instances.

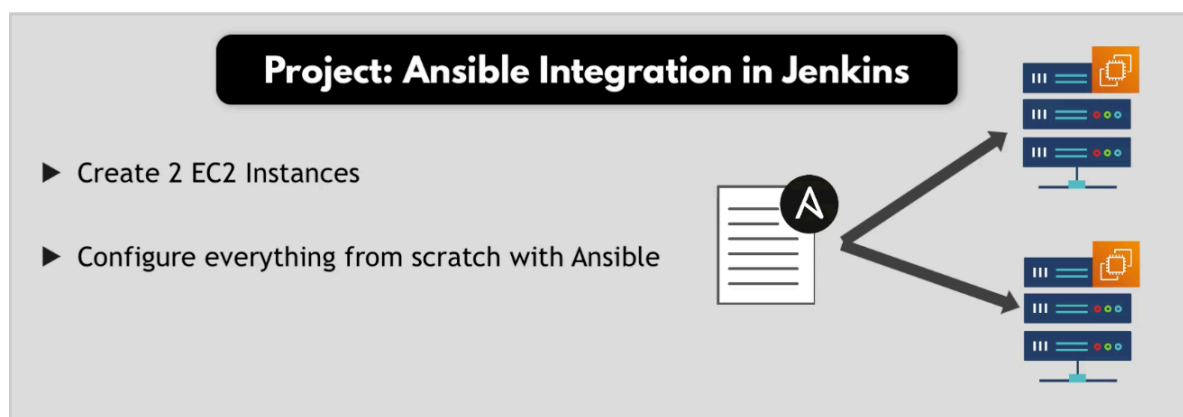


We already have playbook to configure docker and docker-compose called 'deploy-docker-new-user.yaml'

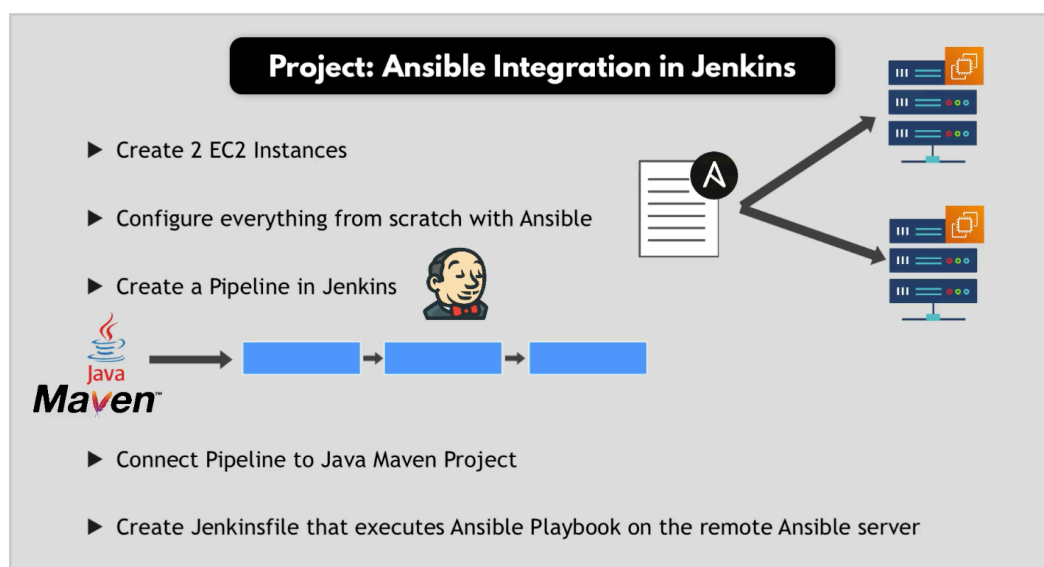
```
Users > astrid > PythonProject > Ansible > deploy-docker-new-user.yaml
Project Files
~/PythonProject/Ansible
> .idea
> .terraform
> .venv
> path
.gitignore
.terraform.lock.hcl
ansible.cfg
deploy-docker.yaml
deploy-docker-new-user.yaml
deploy-nexus.yaml
deploy-node.yaml

17 - name: Install Docker
18   hosts: tag_Name_prod_server
19   become: yes
20   tasks:
21     - name: Make sure Docker is installed
22       yum:
23         name: docker
```

So we will create two EC2 instances and Ansible will configure them

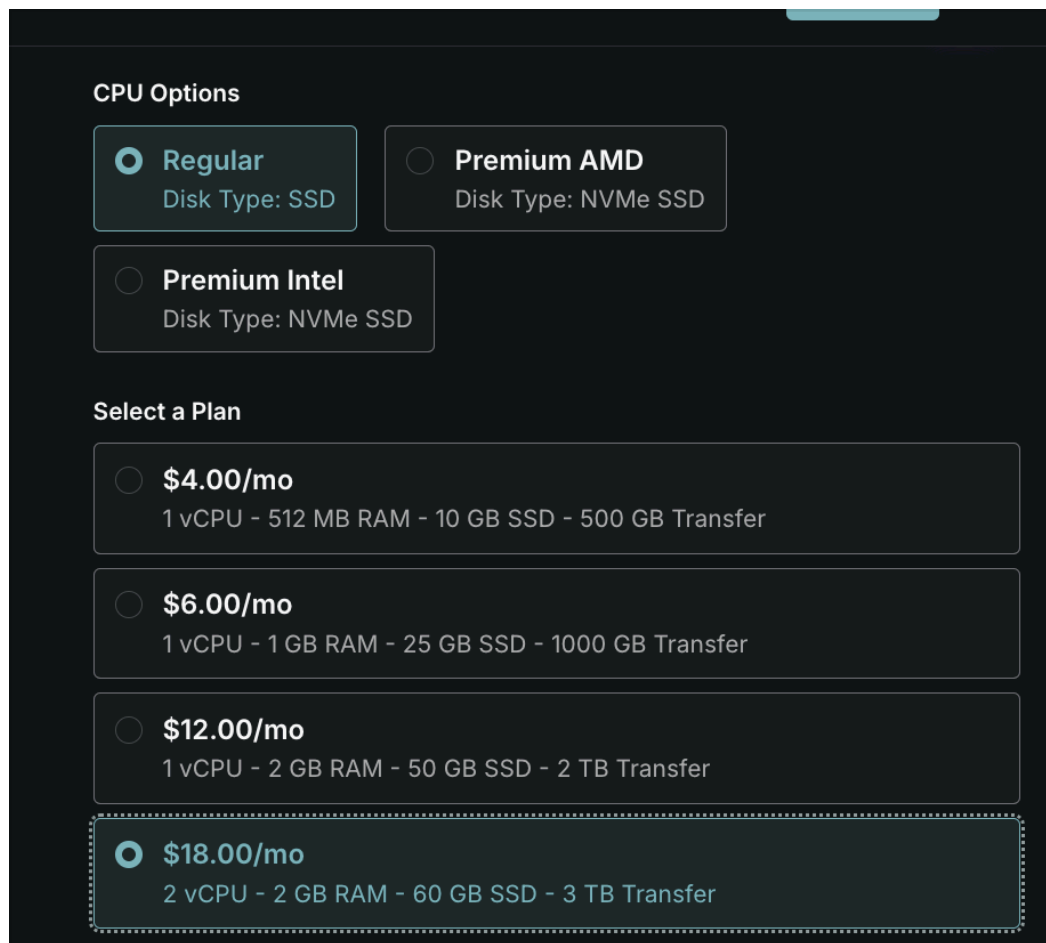


Then we will create a pipeline in Jenkins and connect the pipeline to a Java Maven project and finally create a Jenkinsfile that executes Ansible Playbook on the remote Ansible server



Prepare Ansible Control Node

Create droplet for Ansible -> the ansible control node



The screenshot shows the 'CPU Options' and 'Select a Plan' sections of the DigitalOcean droplet creation interface. Under 'CPU Options', the 'Regular' option is selected with a disk type of 'SSD'. Under 'Select a Plan', the '\$18.00/mo' plan is selected, which provides 2 vCPU, 2 GB RAM, 60 GB SSD, and 3 TB Transfer.

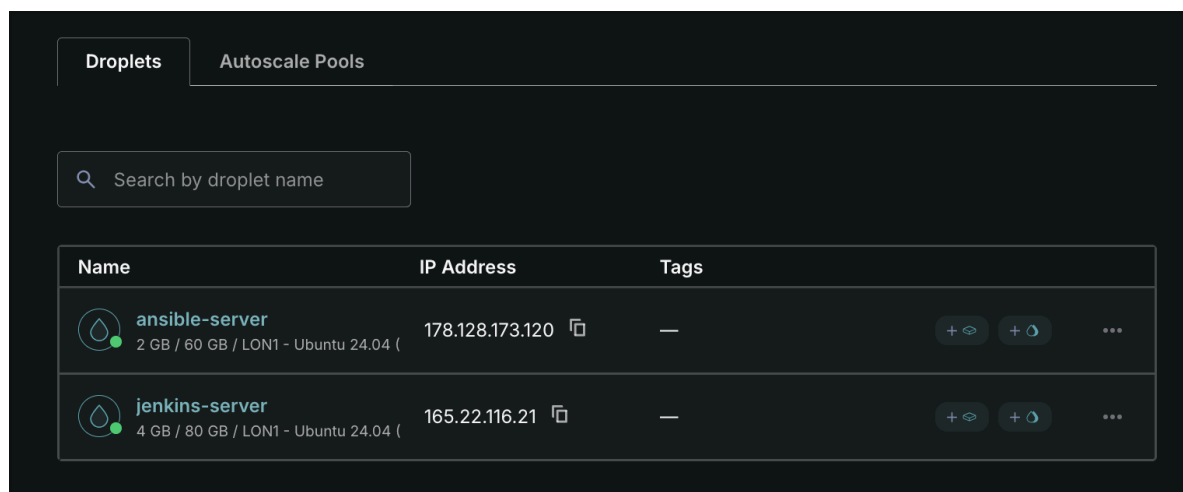
CPU Options

- ☒ **Regular**
Disk Type: SSD
- ☐ **Premium AMD**
Disk Type: NVMe SSD
- ☐ **Premium Intel**
Disk Type: NVMe SSD

Select a Plan

- ☐ **\$4.00/mo**
1 vCPU - 512 MB RAM - 10 GB SSD - 500 GB Transfer
- ☐ **\$6.00/mo**
1 vCPU - 1 GB RAM - 25 GB SSD - 1000 GB Transfer
- ☐ **\$12.00/mo**
1 vCPU - 2 GB RAM - 50 GB SSD - 2 TB Transfer
- ☒ **\$18.00/mo**
2 vCPU - 2 GB RAM - 60 GB SSD - 3 TB Transfer

Server is ready



The screenshot shows the 'Droplets' tab in the DigitalOcean dashboard. It displays a table with two droplets: 'ansible-server' and 'jenkins-server'. The 'ansible-server' droplet has an IP address of 178.128.173.120 and is running Ubuntu 24.04. The 'jenkins-server' droplet has an IP address of 165.22.116.21 and is also running Ubuntu 24.04.

Name	IP Address	Tags
 ansible-server 2 GB / 60 GB / LON1 - Ubuntu 24.04 (	178.128.173.120 	—
 jenkins-server 4 GB / 80 GB / LON1 - Ubuntu 24.04 (	165.22.116.21 	—

Now ssh into the server and install ansible on it

-> ssh **root@178.128.173.120**

-> apt update

```
root@ansible-server:~# apt update
Hit:1 http://mirrors.digitalocean.com/ubuntu noble/universe amd64 Packages
Get:2 http://mirrors.digitalocean.com/ubuntu noble/main amd64 Packages
```

-> ansible

```
root@ansible-server:~# ansible
Command 'ansible' not found, but can be installed with:
apt install ansible-core
root@ansible-server:~#
```

-> apt install ansible-core

```
apt install ansible-core
root@ansible-server:~# apt install ansible-core
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ansible python3-argcomplete python3-dnspython python3-kerberos
  python3-requests-ntlm python3-resolvelib python3-selinux python3-yaml
Suggested packages:
  cowsay sshpass python3-trio python3-aioreactive python3-h2 python3-jinja2
The following NEW packages will be installed:
  ansible ansible-core python3-argcomplete python3-dnspython python3-
  python3-passlib python3-requests-ntlm python3-resolvelib python3-yaml
0 upgraded, 15 newly installed, 0 to remove and 95 not upgraded
Need to get 19.5 MB of archives.
After this operation, 315 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://mirrors.digitalocean.com/ubuntu noble/universe amd64 ansible
Get:2 http://mirrors.digitalocean.com/ubuntu noble/main amd64 python3-argcomplete
```

```
Setting up python3-winrm (0.4.3-2) ...
Setting up ansible (9.2.0+dfsg-0ubuntu5) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ansible-server:~#
```

Also, we need to install a python module for AWS -> boto3, ansible is based on python. Some of the tasks we run in ansible need the python module for AWS which ansible will use in the background



Why Ansible needs it

Ansible itself is a Python program, but it doesn't know how to talk to any specific API (AWS, Kubernetes, etc.) natively. Each provider's modules (e.g. the `amazon.aws` / `community.aws` collections) are thin Python wrappers that, under the hood, call `boto3` — AWS's official Python SDK — to actually make the HTTP requests to the AWS API (create an EC2 instance, read a VPC, etc.).

So when a task like `amazon.aws.ec2_instance` runs, the sequence is:

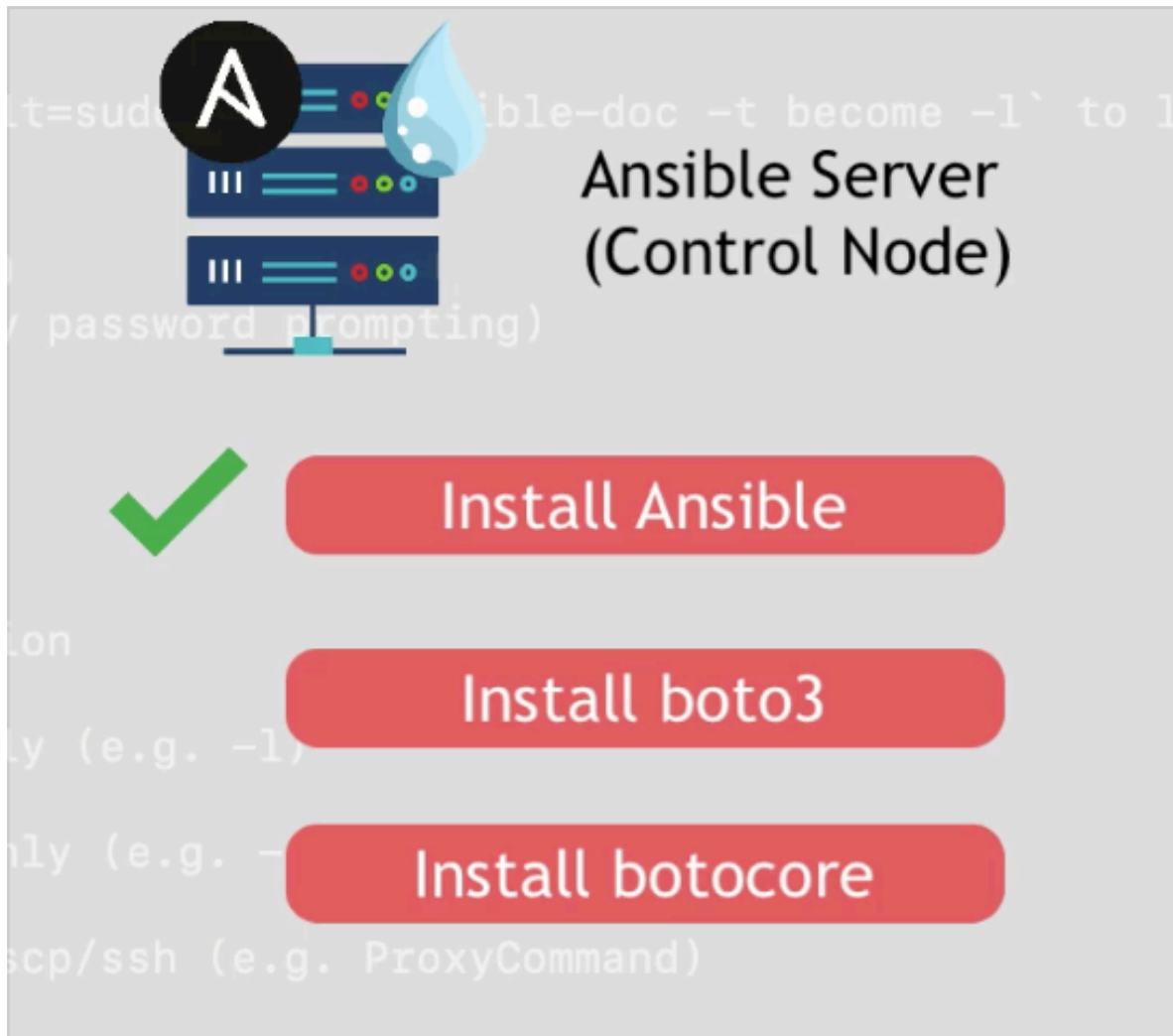
Ansible task → AWS collection module (Python) → `boto3` → AWS API

If `boto3` isn't installed in the Python environment Ansible is using, the module fails immediately with an error like `"msg": "boto3 required for this module"` — it's not an Ansible bug, it's a missing dependency for the underlying library the

module calls into.

(You may also see botocore listed alongside it — that's the lower-level library boto3 itself is built on, handling auth, request signing, retries. Installing boto3 usually pulls botocore in as a dependency automatically.)

As we will configure EC2 instances with Ansible we will use boto3 and botocore, so we need to install them



-> apt install python3-boto3

```
root@ansible-server:~# apt install python3-boto3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-boto3 is already the newest version (1.34.46+dfsg-1ubuntu1).
python3-boto3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 95 not upgraded.
root@ansible-server:~#
```

apt didn't need to do anything about it this run.

Why

python3-boto3 is a Debian/Ubuntu package that depends on python3-botocore (declared in its package metadata). apt resolves and installs dependencies automatically the first time you install a package — but in your case, boto3 was "already the newest version," meaning it (and therefore botocore, pulled in as its dependency) was already installed before you ran this command, likely baked into the droplet image or installed earlier. Since nothing needed to change, apt's output only reports "0 upgraded, 0 newly installed, 0 to remove" — it doesn't re-announce dependencies that are already satisfied.

-> apt show python3-boto3

```
root@ansible-server:~# apt install python3-boto3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-boto3 is already the newest version (1.34.46+dfsg-1ubuntu1).
python3-boto3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 95 not upgraded.
root@ansible-server:~# apt show python3-boto3
Package: python3-boto3
Version: 1.34.46+dfsg-1ubuntu1
Priority: optional
Section: python
Source: python-boto3
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Original-Maintainer: Debian Cloud Team <debian-cloud@lists.debian.org>
Bugs: https://bugs.launchpad.net/ubuntu/+filebug
Installed-Size: 1062 kB
Depends: python3-botocore (>= 1.31.49), python3-requests, python3-s3transfer (>= 0.1.10), python3-jmespath, python3:any
Homepage: https://github.com/boto/boto3
Download-Size: 72.5 kB
APT-Manual-Installed: yes
APT-Sources: http://mirrors.digitalocean.com/ubuntu noble/main amd64 Packages
Description: Python interface to Amazon's Web Services - Python 3.x
  Boto is the Amazon Web Services interface for Python. It allows developers
  to write software that makes use of Amazon services like S3 and EC2. Boto
  provides an easy to use, object-oriented API as well as low-level direct
  service access.
```



Now we need AWS credentials:

boto3 also needs AWS credentials to actually authenticate — same as the AWS CLI. Either:

- `~/.aws/credentials` on the droplet, or
- environment variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`), or
- an IAM role if the droplet were an EC2 instance (not applicable here since it's a Droplet, i.e. DigitalOcean, not AWS).

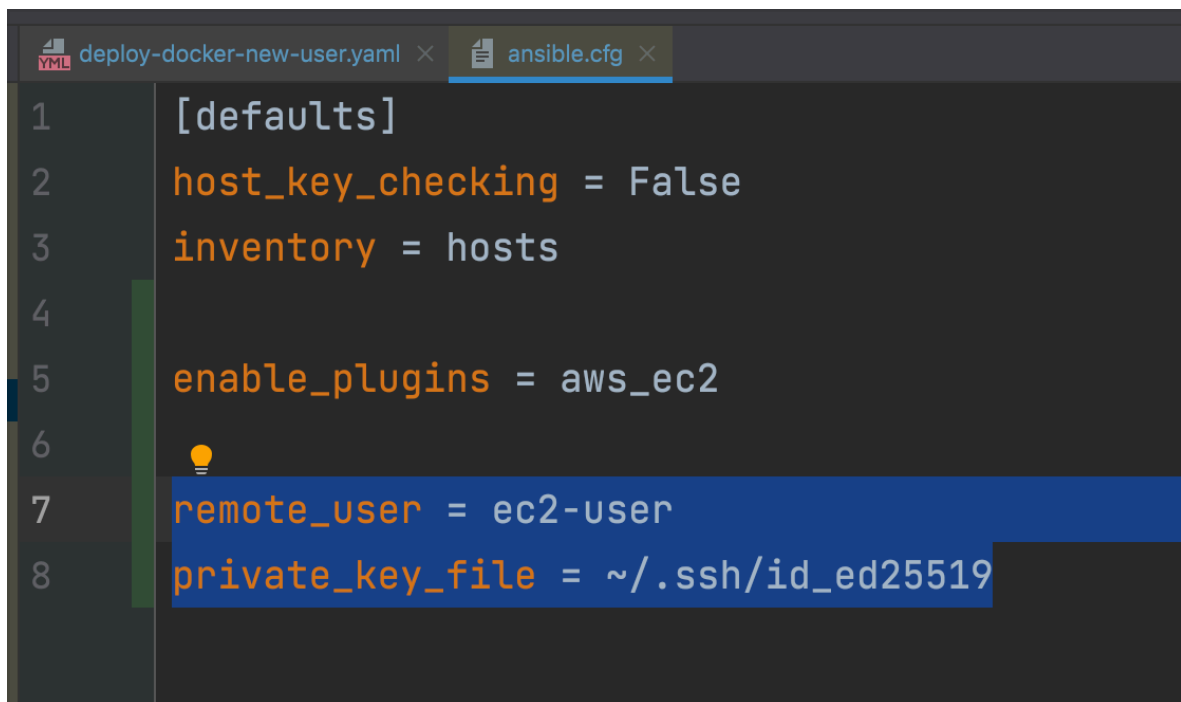
We will be using a dynamic inventory for the docker playbook

```
! inventory_aws_ec2.yaml U x  ! deploy-docker-new-user.yaml U
1  ---
2  plugin: aws_ec2
3  regions:
4  - eu-central-1
5  keyed_groups:
6  - key: tags
7    prefix: tag
8  - key: instance_type
9    prefix: instance_type
10
```

So instead of hardcoding the EC2 instances IP addresses we get them dynamically from AWS and we need AWS credentials for that dynamic inventory to work.

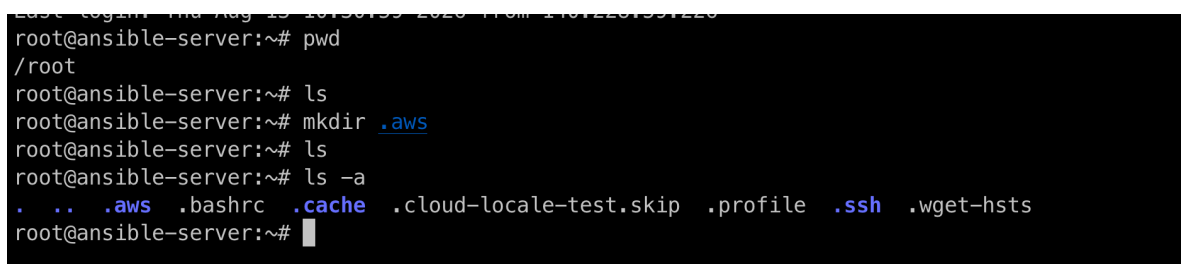
The inventory will try to fetch the EC2 instances and for that it needs to authenticate (when we run this in my laptop in Module 15 - chapter 19 the auth info was already there due to -> aws configure - which wrote my access key/secret into **~/.aws/credentials**) but in this case the droplet doesn't have the credentials stored anywhere.

Don't confuse the AWS credentials with remote_user and private_key_file from Module 15 - chapter 19 notes, that configuration is to ssh into each host not to authenticate to AWS.



```
1 [defaults]
2 host_key_checking = False
3 inventory = hosts
4
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/.ssh/id_ed25519
```

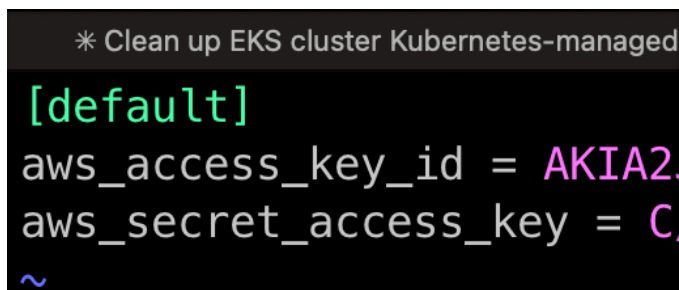
So we need to create the .aws folder inside the ansible server
-> mkdir .aws



```
root@ansible-server:~# pwd
/root
root@ansible-server:~# ls
root@ansible-server:~# mkdir .aws
root@ansible-server:~# ls
root@ansible-server:~# ls -a
.  ..  .aws  .bashrc  .cache  .cloud-locale-test.skip  .profile  .ssh  .wget-hsts
root@ansible-server:~#
```

Now let's create the file credentials inside .aws directory
-> cd .aws
-> vim credentials

And I can copy the contents of my laptop's credentials file into this file:



```
* Clean up EKS cluster Kubernetes-managed

[default]
aws_access_key_id = AKIA2
aws_secret_access_key = C
~
```

```

root@ansible-server:~# cd .aws/
root@ansible-server:~/.aws# vim credentials
root@ansible-server:~/.aws# cat credentials
[default]
aws_access_key_id = [REDACTED]
aws_secret_access_key = [REDACTED]
root@ansible-server:~/.aws# █

```

So we have boto library ready and the credentials ready for when we need it to use dynamic inventory because the plugin will need to authenticate with AWS

```

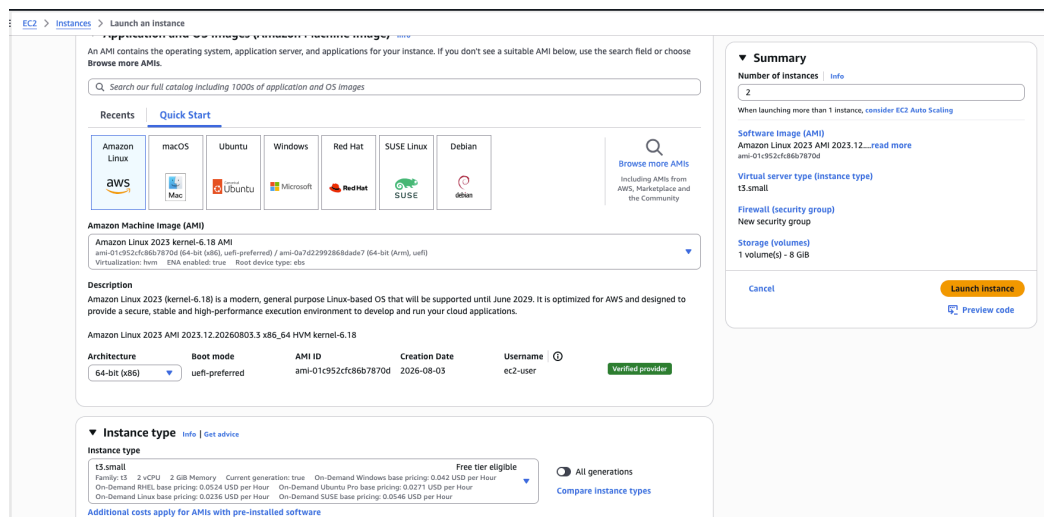
root@ansible-server:~# ls .aws/credentials
.aws/credentials
root@ansible-server:~# █

```

Ansible server is ready ❤️

Create two EC2 Instances to be managed by Ansible

We will launch the EC2 instances manually from the AWS console.



And we will create a new key pair, remember that ansible connects to the instances using the ssh key

Create key pair

Key pair name

Key pairs allow you to connect to your instance securely.

ansible-jenkins

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA
RSA encrypted private and public key pair

☐ ED25519
ED25519 encrypted private and public key pair

Private key file format

☒ .pem
For use with OpenSSH

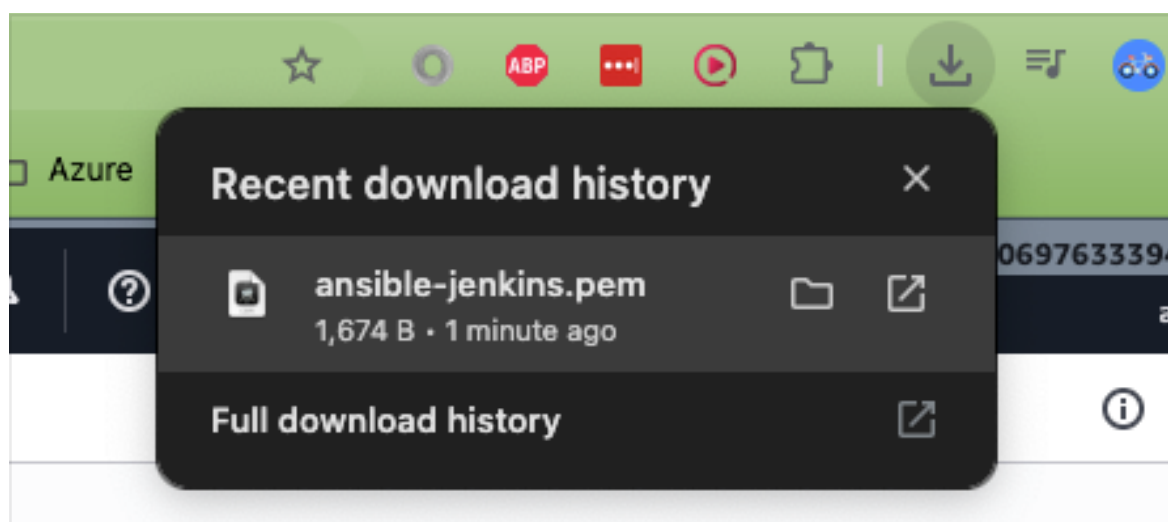
☐ .ppk
For use with PuTTY

 When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** [Learn more](#)

Cancel

Create key pair

And we download the .pem file



And we launch the 2 instances

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

ansible-jenkins

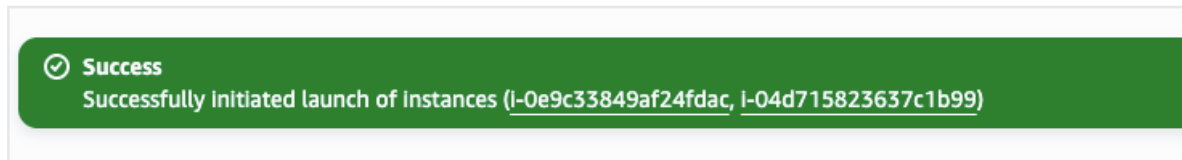
Create new key pair

1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code



EC2 > Instances

2

Instances (2) Info

Last updated less than a minute ago

Connect

Instance state

Ac

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐		i-04d715823637c1b99	Running	t3.small	Initializing	eu-west-2c	ec2-18-170-46-239.eu-...	18.170.46.239
☐		i-0e9c33849af24fdac	Running	t3.small	Initializing	eu-west-2c	ec2-18-170-50-173.eu-...	18.170.50.173

So we have two EC2 instances and the private key to connect to those two instances.

This is all we need to configure for the EC2 instances, the rest of the configuration will be done by Ansible.